

Chapter 5

5.1 Microcontrollers

5.1.1 What is a Microcontroller?

A microcontroller is a programmable integrated chip (IC). It has different ports which can be controlled using a program written by the user. These ports control the connected circuitry according to the uploaded program. A typical microcontroller has a microprocessor unit (MPU), memory, and some peripherals.

5.1.2 What Do Microcontrollers Do?

A microcontroller board can sense, monitor and respond to various events, behaviors or input signals that it receives from connected components.

A microcontroller, for example, might be programmed to push a specific type of output signal or behavioral control in response to certain input conditions. This could include the execution of tasks such as:

- Turning ON an LED or OLED display in response to touch-based user demand.
- Playing lights and sounds in temperature-sensing applications or other varieties of alarms and warning systems.
- Responding to the need for a motor to switch on or off in a pump or other mechanical device.
- Adjusting tilt, balance, and velocity in gyroscope or accelerometer-based applications.

5.1.3 Microcontroller vs Microprocessors

A microprocessor does not contain any memory (RAM or ROM) or I/O ports. The instruction set telling a standalone microprocessor how to execute a given function are generally stored externally. In a microcontroller, all these various components - including the simplified processor - are combined into a single self-contained unit.

Performance-wise, microcontrollers and microprocessors are differentiated as:

Microcontroller

- is an entirely self-contained unit that contains a very simple CPU or microprocessor.
- is used for a single specific application, as pre-programmed by the user
- is not especially powerful in performance terms; typically, it only draws a small amount of power and contain little in terms of integrated data storage capacity.
- needs to be programmed by the operator to perform any meaningful role.
- can not operate outside of their specifically programmed code.
- is generally meant for use in specific devices or appliances designed to perform one task repeatedly.

Microprocessor

- is much more complex and versatile in terms of function range, and intended for use in more general computing (as opposed to in specialized one-task devices).
- have much faster processor (clock) speeds than MCUs, often measured in gigahertz (GHz) rather than Hz.
- is challenging and expensive to manufacture, unlike relatively simple and cheap microcontrollers.
- require far more external components (RAM, I/O ports, data storage, EEPROM or flash memory) to operate, none of which are integrated into the microprocessor and must be connected separately.
- have a considerably higher power draw and are subsequently much less cost-effective to run continually.

Chapter 5

Common Microcontrollers

Some commonly used microcontrollers are:

1. 8086 Microcontrollers
2. AVR Microcontrollers
3. Arduino Boards
4. STM
5. ESP
6. Raspberry Pi

Microcontrollers for IoT

Most widely used microcontrollers for development of IoT systems are:

1. ESP8266
2. ESP32
3. Raspberry Pi

5.2 Arduino

Arduino is an open-source embedded systems platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter message - and turn it into an output - activating a motor, turning on an LED, publishing something online. You can tell your board what to do by sending a set of instructions to the microcontroller on the board. You need to use the Arduino programming language (based on Wiring), and the Arduino Software (IDE) to start working on Arduino.

5.2.1 Arduino UNO

Arduino UNO is a microcontroller board based on the ATmega328P (datasheet). It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator (CSTCE16M0V53-R0), a USB connection, a power jack, an ICSP header and a reset button. It contains everything needed to support the

microcontroller; simply connect it to a computer with a USB cable or power it with an AC-to-DC adapter or battery to get started. Pin configuration of Arduino UNO is shown below:

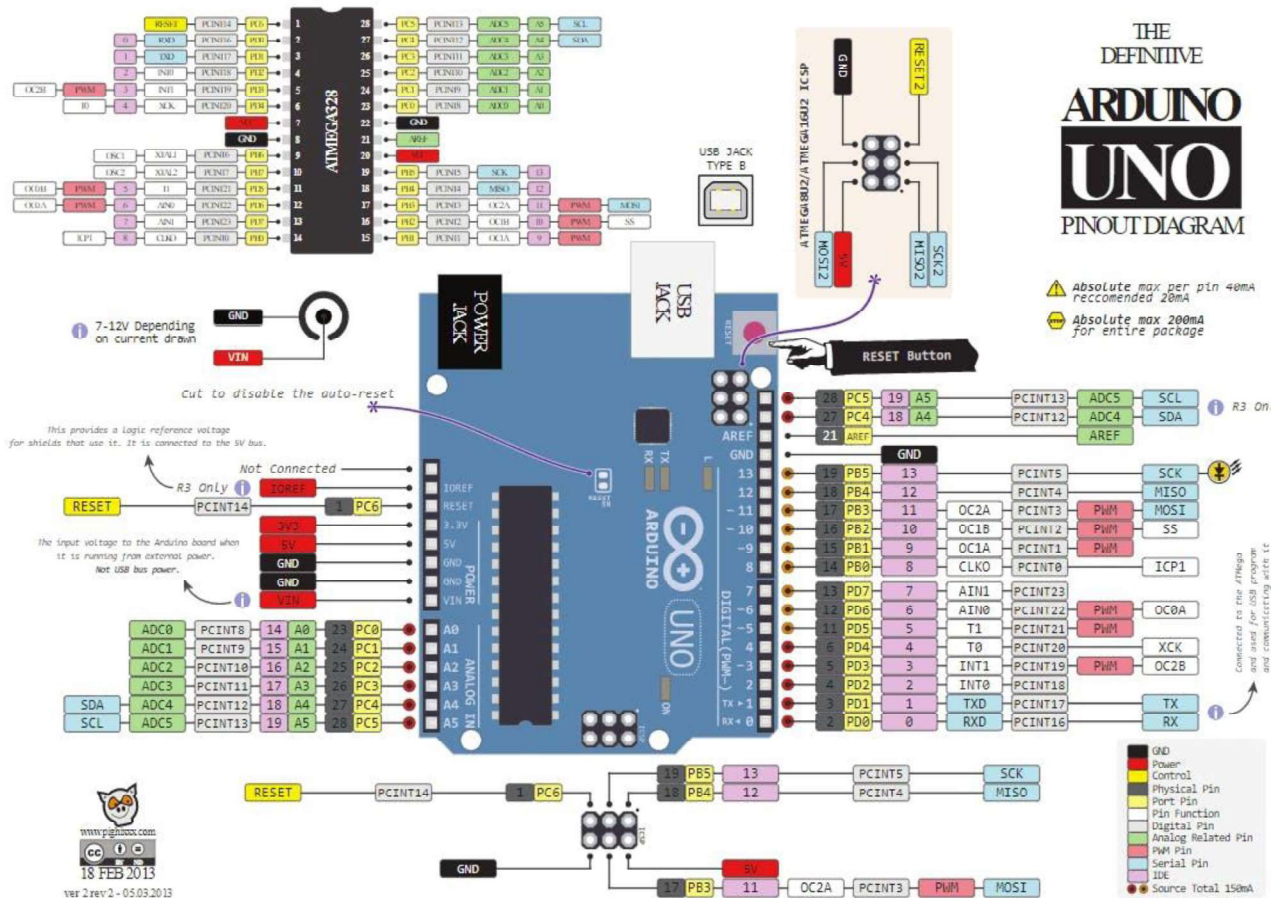


Fig 5.1 Pin Configuration of Arduino UNO

5.3 Arduino IDE

- Arduino IDE is an open-source software, designed by Arduino.cc and mainly used for writing, compiling & uploading code to almost all Arduino Modules.
- It is an official Arduino software, making code compilation easy.
- It is available for all operating systems i.e., MAC, Windows, Linux and runs on the Java Platform that comes with inbuilt functions and commands that play a vital role in debugging, editing and compiling the code.

Chapter 5

- A range of Arduino modules are available including Arduino Uno, Arduino Mega, Arduino Leonardo, Arduino Micro and many more.
- Each of them contains a microcontroller on the board that is actually programmed and accepts the information in the form of code.
- The main code, also known as a sketch, created on the IDE platform will ultimately generate a Hex File (executable file containing hex codes) which is then transferred and uploaded in the controller on the board.
- The IDE environment mainly contains two basic parts: Editor and Compiler. The former is used for writing the required code and the latter is used for compiling and uploading the code into the given Arduino Module.
- This environment supports both C and C++ languages.

5.3.1 Installing the Arduino IDE

1. Visit <http://www.arduino.cc/en/main/software> to download the latest Arduino IDE version for your computer's operating system. There are versions for Windows, Mac, and Linux systems. At the download page, click on the “Windows Installer” option for the installation.
2. Save the .exe file to your hard drive.
3. Open the .exe file.
4. Click the button to agree to the licensing agreement:

Do you know?

Instruction set of every microcontroller is in Assembly language which is converted to machine language instructions while compiling.

Point to Ponder

What is the programming interface of a microcontroller?



Fig 5.2 Arduino setup: License agreement

5. Decide which components to install, then click “Next”:



Fig 5.3 Arduino Setup: Installation options

6. Select which folder to install the program to, then click “Install”:

Chapter 5

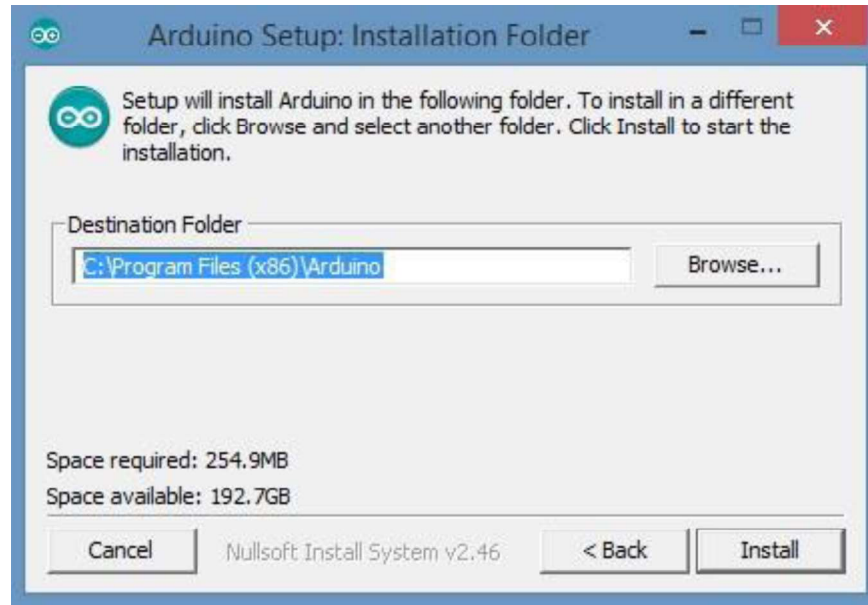


Fig 5.4 Arduino Setup: Installation folder

7. Wait for the program to finish installing, then click “Close”.
8. Now find the Arduino shortcut on your Desktop and click on it. The IDE will open up and you’ll see the code editor.

Libraries

Libraries are a collection of code that makes it easy for you to connect to a sensor, display, module, etc. For example, the built-in ‘LiquidCrystal’ library makes it easy to talk to character LCD displays. There are hundreds of additional libraries available on the Internet for download. To use the additional libraries, you will need to install them.

How to Install a Library?

Using the Library Manager

To install a new library into your Arduino IDE you can use the Library Manager (available from IDE version 1.6.2 and above). Open the IDE and click to the "Sketch" menu and then *Include Library > Manage Libraries*:

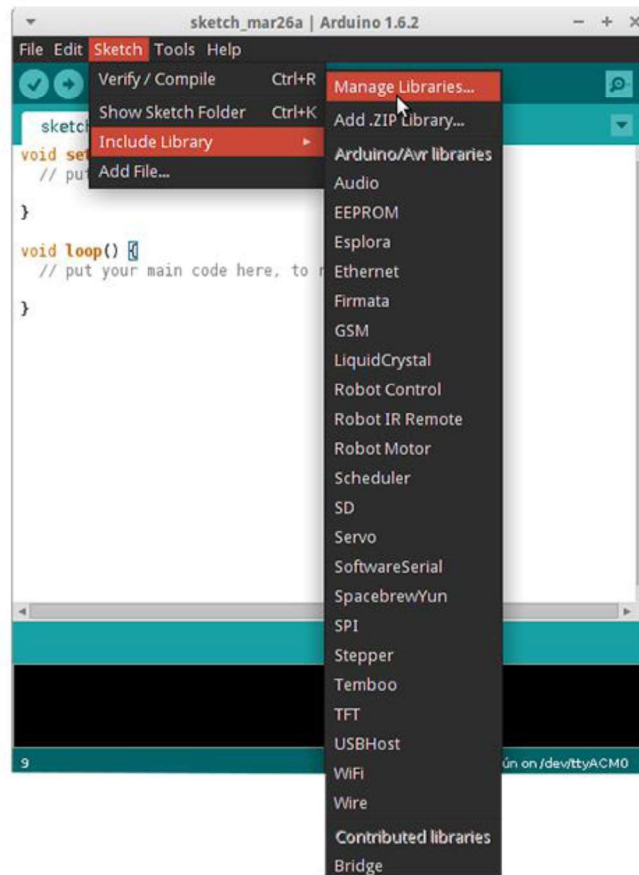


Fig 5.5 Arduino IDE: Include Library

Then the Library Manager will open and you will find a list of libraries that are already installed or ready for installation. In this example we will install the Bridge library. Scroll the list to find it, click on it, then select the version of the library you want to install. Sometimes only one version of the library is available:

Chapter 5

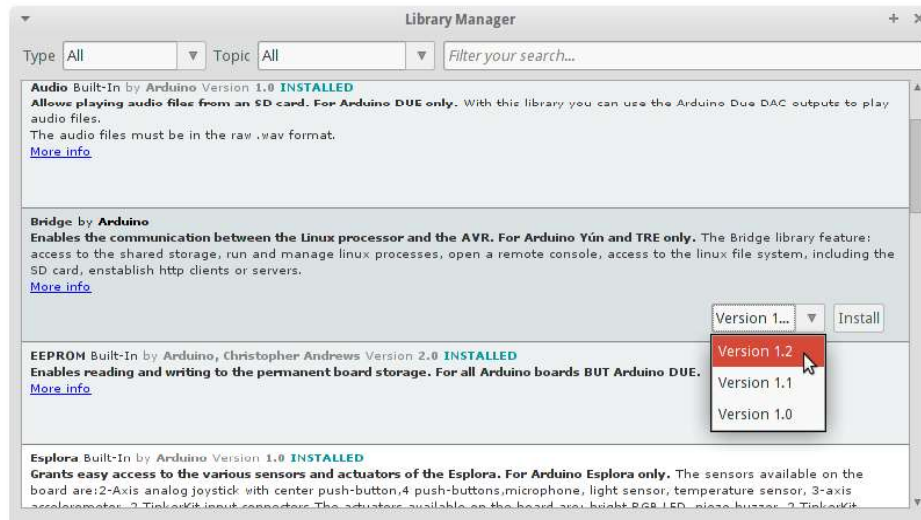
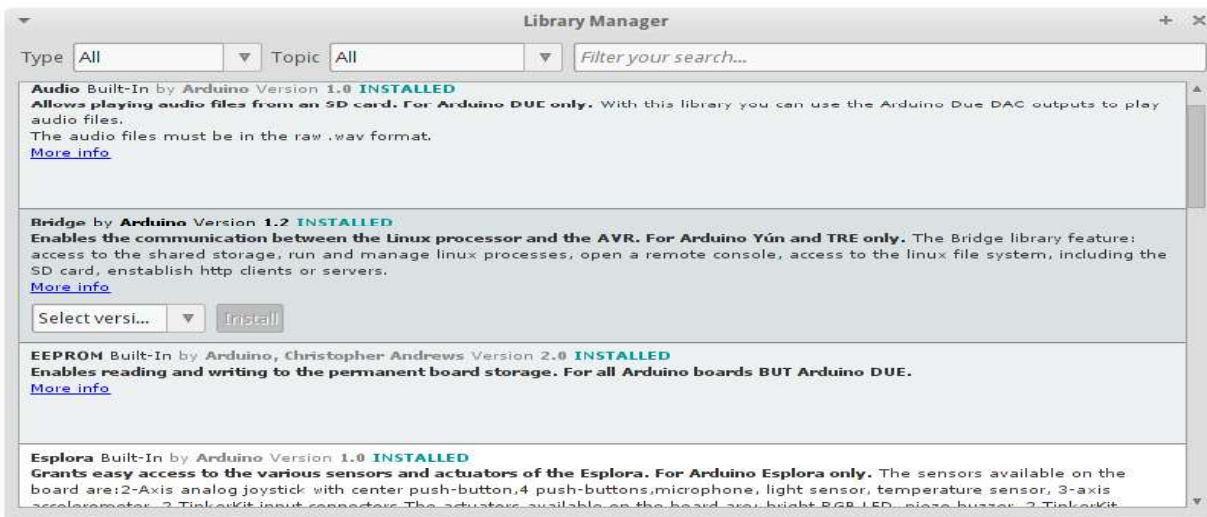


Fig 5.6 Arduino Setup: Library Manager

Finally click on install and wait for the IDE to install the new library. Downloading may take time depending on your connection speed. Once it has finished, an *Installed* tag should appear next to the Bridge library. You can close the library manager:



Teacher Notes:

- Tell students about the available interfaces on Arduino UNO.

You can now find the new library available in the *Sketch > Include Library* menu. If you want to add your own library to Library Manager, follow these instructions.

Importing a .zip Library

Libraries are often distributed as a ZIP file or folder. The name of the folder is the name of the library. Inside the folder will be a .cpp file, a .h file and often a keywords.txt file, examples folder, and other files required by the library. Starting with version 1.0.5, you can install 3rd party libraries in the IDE. Do not unzip the downloaded library, leave it as is.

In the Arduino IDE, navigate to *Sketch > Include Library > Add .ZIP Library*. At the top of the drop-down list, select the option to "Add .ZIP Library":

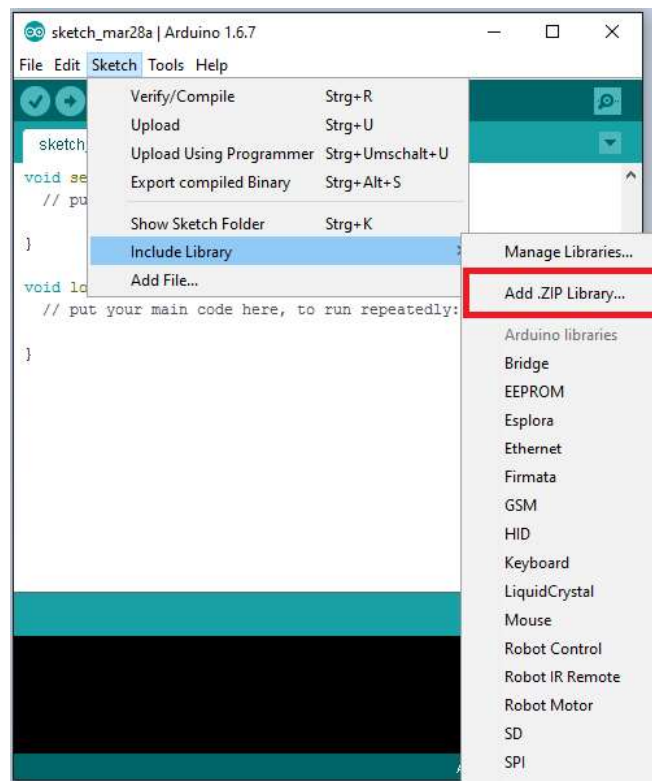


Fig 5.8 Arduino IDE: Adding .zip file of library

You will be prompted to select the library you would like to add. Navigate to the .zip file's location and open it:

Chapter 5

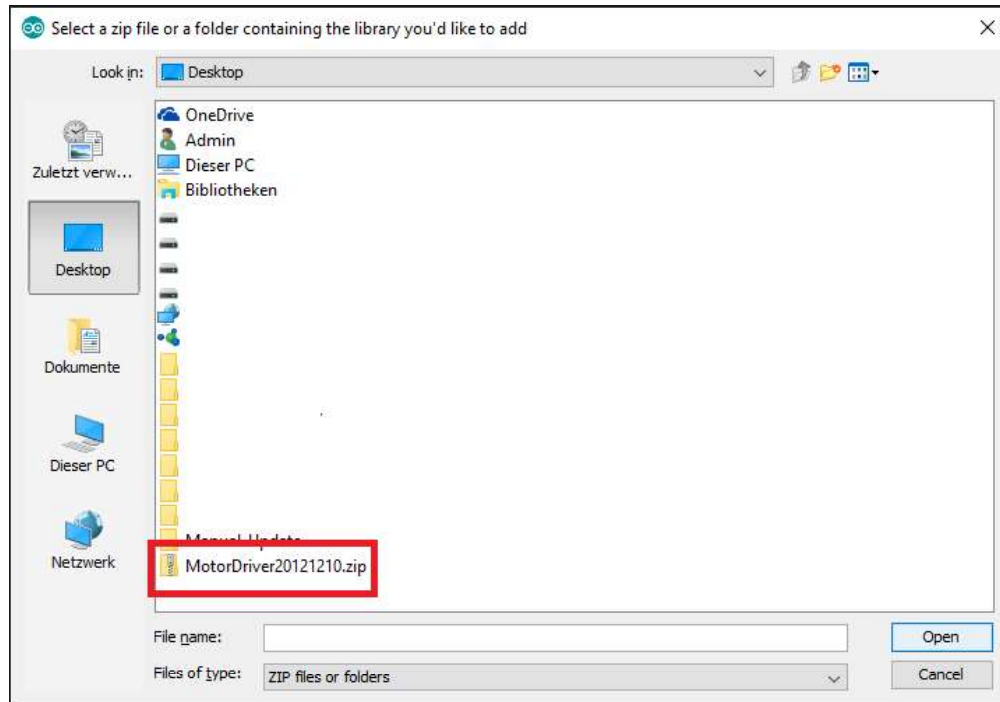


Fig 5.9 Arduino Setup: Selecting .zip file

Return to the *Sketch > Include Library menu.* menu. You should now see the library at the bottom of the drop-down menu. It is ready to be used in your sketch. The zip file will have been expanded in the *libraries* folder in your Arduino sketches directory.

5.4 Connecting an Arduino with Computer System

1. Connect Arduino to your computer

USB-Mini cable is used to connect Arduino to your computer. Different types of USB connectors are shown in figure:



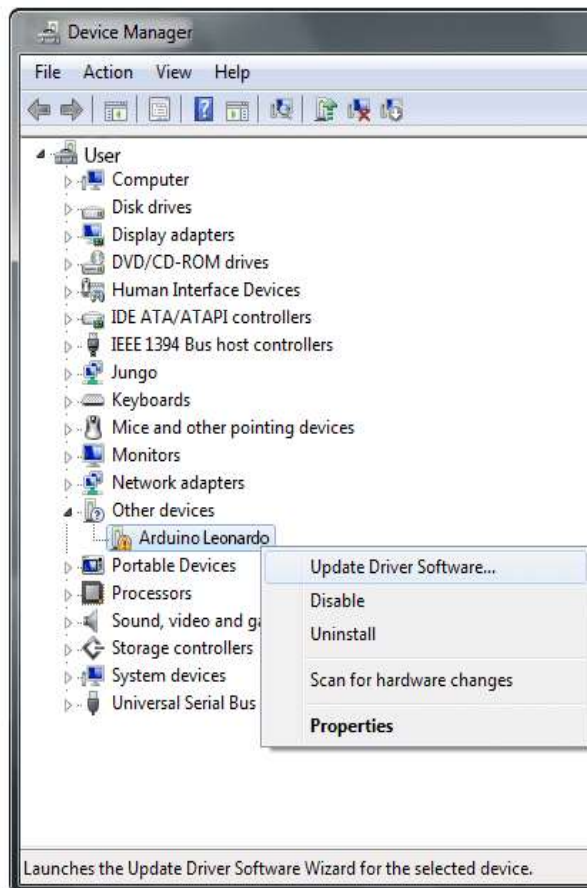
Fig 5.10 USB cables

2. Install/update USB-Serial Driver

The following instructions are for Windows 7, Vista and 10. They are also valid for Windows XP, with small differences in the dialog windows. In the following instruction only the Leonardo board will be mentioned, but the same procedure is valid for all the Arduino boards.

Plug in your board and wait for Windows to begin its driver installation process. If the installer does not launch automatically, navigate to the Windows Device Manager (Start>Control Panel>Hardware) and find the Arduino listing. Right click and choose **Update driver**:

Note: The following figure shows driver for Arduino Leonardo. Process of updating the driver is same for all boards.



Interesting Information

- A device driver is a software component that allows a hardware device to communicate with the operating system of a computer. Drivers allow an operating system to correct interpret and implements the signals that come from the hardware device.

Fig 5.11 Arduino Setup: Device manager: Update Driver

Chapter 5

In the next screen, click "Browse my computer for driver software", and click **Next**.

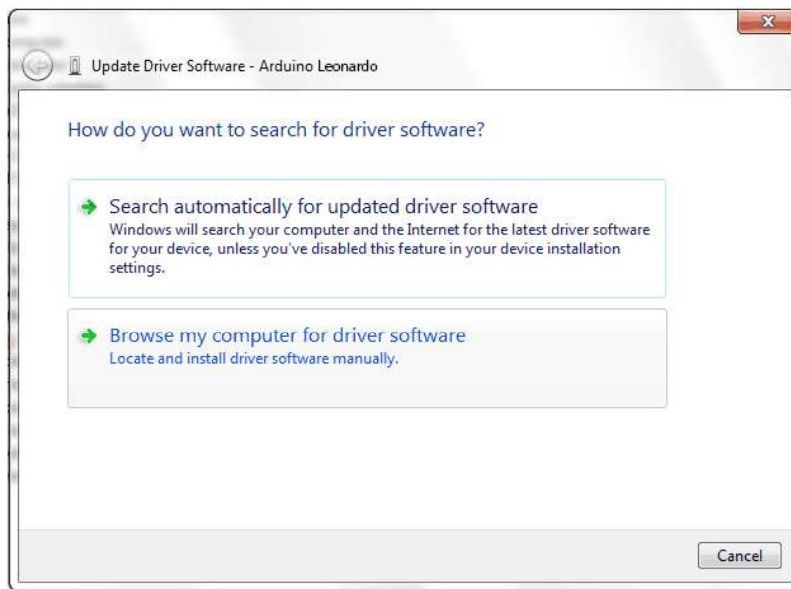


Fig 5.12 Arduino Setup: Update Driver

Click the **Browse** button. Another dialog appears: navigate to the folder with the Arduino software that you just downloaded. Select the **drivers** folder and click **OK**, then click **Next**.



Fig 5.13 Arduino Setup: Select Driver

You will receive a notification that the board has not passed Windows Logo testing. Click on the button **Continue Anyway**.

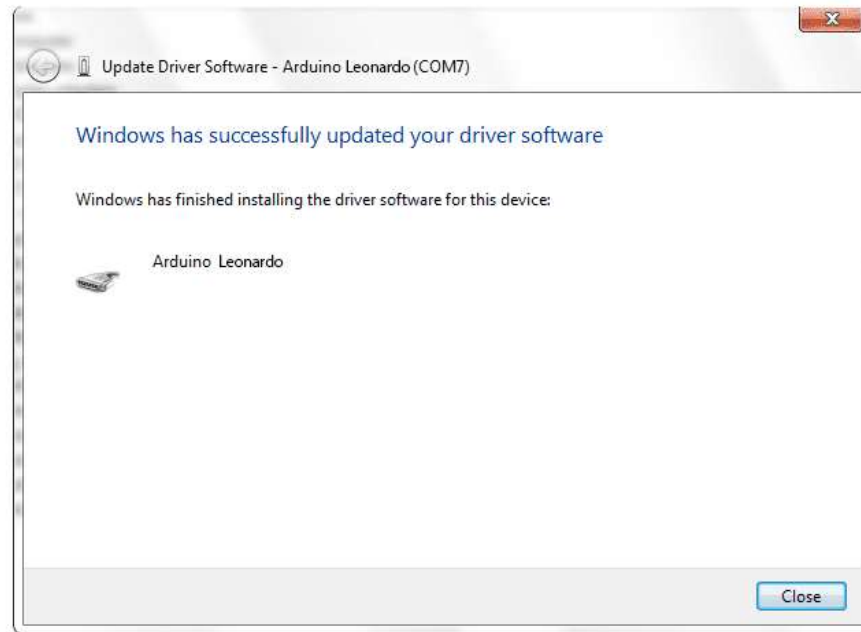


Fig 5.14 Arduino Setup: Driver installed

After a few moments, a window will tell you the wizard has finished installing software for Arduino Leonardo. Press the **Close** button.

5.5 Programming Arduino

Open the LED blink example sketch: File > Examples > 01.Basics > Blink.

Chapter 5

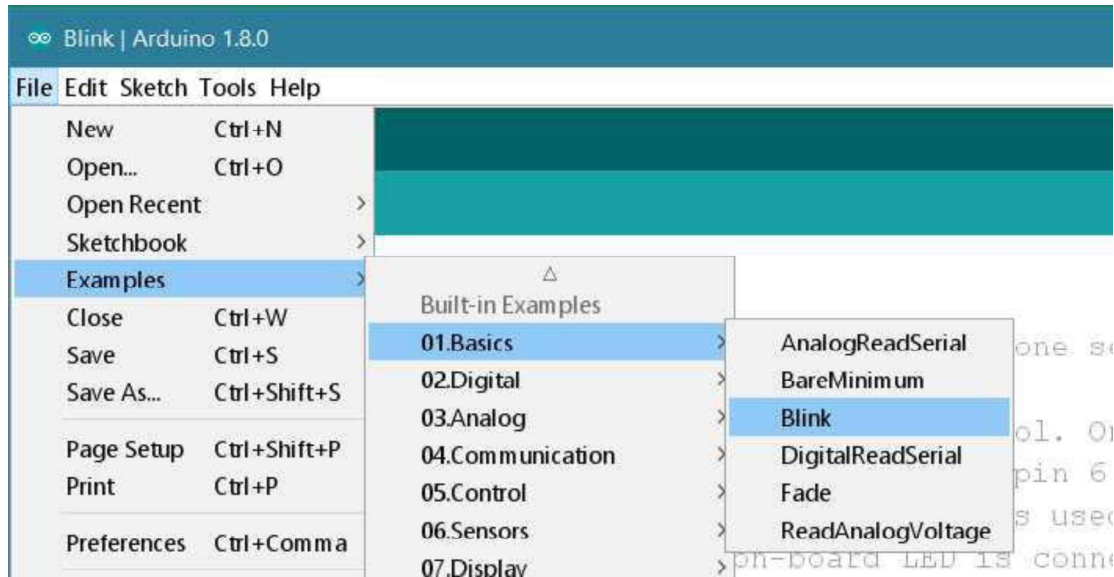


Fig 5.15 Arduino IDE: Blink Example

You'll need to select the entry in the **Tools > Board** menu that corresponds to your Arduino board.

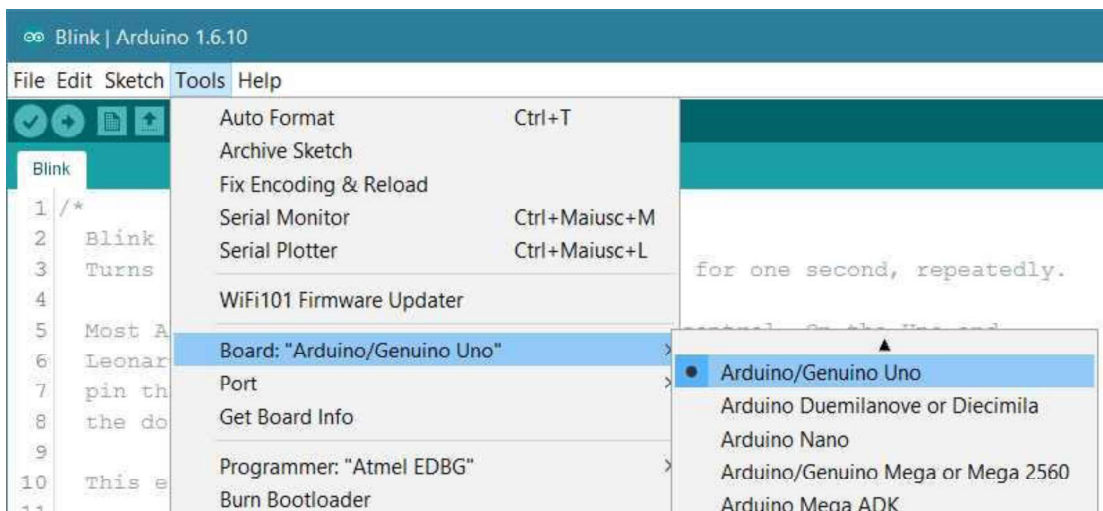


Fig 5.16 Arduino IDE: Select Board

Select the serial device of the board from the Tools | Serial Port menu. This is likely to be **COM3** or higher (**COM1** and **COM2** are usually reserved for hardware serial ports).

To find out, you can disconnect your board and re-open the menu; the entry that disappears should be the Arduino board. Reconnect the board and select that serial port.

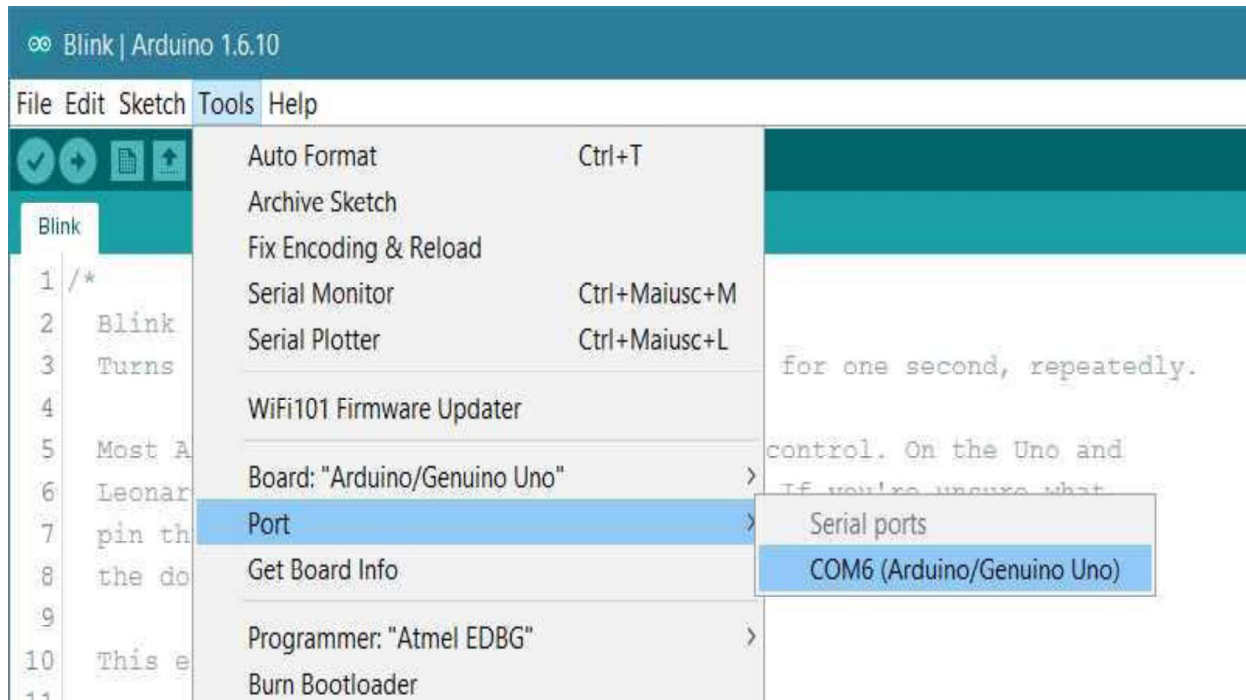


Fig 5.17 Arduino IDE: Select Port

Hardware Setup

Place the LED sensor in the breadboard. Connect a 220-ohm resistor to its anode and connect its pin to GND pin of Arduino. Connect a wire from the resistor to corresponding pin on Arduino (digital pin). Connect the Cathode of LED to GND of Arduino.

Chapter 5

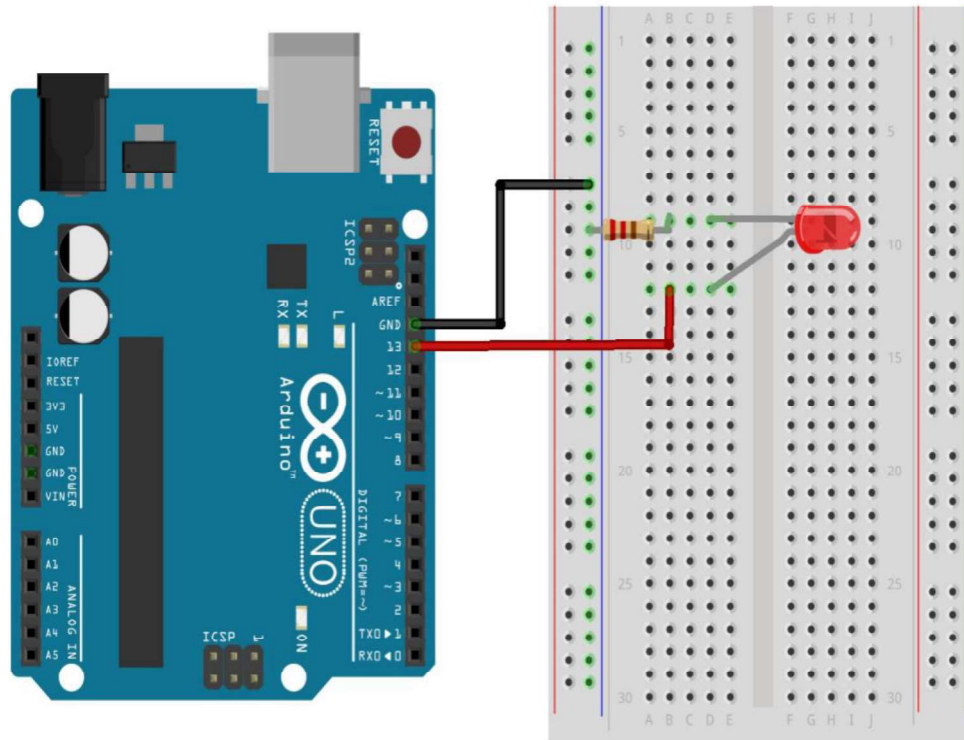


Fig 5.17 Arduino: Hardware Setup for Blink Example

Simply click the "Upload" button in the environment. Wait a few seconds - you should see the RX and TX LEDs on the board flashing. If the upload is successful, the message "Done uploading." will appear in the status bar.

Key Points

- A microcontroller has different ports which can be controlled using a program written by the user.
- Microcontroller contains peripherals i.e. RAM/ROM, I/O ports, EEPROM etc.
- A microprocessor contains no peripherals.
- IDE is used to write a code, compile code, burn the code and debug the program.
- A device driver is necessary to connect arduino with computer.
- Operating voltage of Arduino is 5V.
- Libraries are the ready-made programs/functions which can be added in the existing code according to the need.

Exercise

Multiple Choice Questions

1. A microcontroller contains the following peripherals:
a. RAM & ROM b. I/O ports c. EEPROM d. All of these
2. Which of the following is not a Arduino board:
a. Arduino UNO b. Arduino Leonardo c. Arduino Nano d. None of these
3. Arduino operates at:
a. 5V b. 3.3 V c. 20V d. Both a and b
4. Which of the following IDE is used for Arduino.
a. Keil b. Microsoft Visual Studio c. Eclipse d. Arduino IDE
5. Which interfaces are available on Arduino
a. UART b. SPI c. I2C d. All of these

Write short answer of the following.

1. Define microcontroller.
2. Differentiate between microcontroller and microprocessor.
3. Define IDE.
4. What are libraries and why are they used?
5. Name some arduino based boards.
6. Name some commonly used microcontrollers for IoT.
7. What is a device driver.
8. What do you mean by pin configuration?
9. What are I/O ports?
10. Name some I/O interfaces available on Arduino.

Activites.

1. Install Arduino IDE.
2. Install device driver for Arduino.
3. Add any library in Arduino IDE.
4. Connect an LED with Arduino and blink it using an example program.
5. Explore serial port. Print some text on serial monitor of Arduino IDE.